



Guía de Actividades Práctico-Experimentales Nro. 002

1. Datos Generales

Nombre del Estudiante	Mateo Rojas
Asignatura	Sistemas Distribuidos
Ciclo	6to
Unidad	Uno
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	2
Título de la Práctica	Arquitectura de Sistemas Distribuidos (comunicación cliente-servidor con archivos)
Nombre del Docente	José O. Guamán Q.
Fecha	06 de octubre de 2026 09 de octubre de 2026
Horario	Lunes 10h30 – 11h30 Viernes 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

Comprender la arquitectura de sistemas distribuidos mediante el desarrollo independiente de sus componentes (cliente y servidor).

3. Materiales y reactivos:

Guia practica

Se requiere un sistema que permita crear un servidor y un cliente con un canal de comunicación (archivo de texto), se debe evidenciar que se esta llenado el archivo.

4. Equipos y herramientas

Hardware

- Computador

Software

- Lenguaje de programación

-
- IDE de programación
 - Archivo de texto

5. Procedimiento / Metodología

Parte 1: Desarrollo del SERVIDOR (1 hora 20 minutos)

Este módulo se desarrolla de forma independiente

Actividad

1. Crear un archivo:
servidor.py
2. Crear un archivo auxiliar:
entrada_servidor.txt
3. Implementar un programa que:
 - Revise periódicamente el archivo entrada_servidor.txt.
 - Detecte si existe contenido nuevo.
 - Procese el mensaje:
 - Ejemplo: convertir texto a mayúsculas o invertirlo.
 - Escriba el resultado en:
salida_servidor.txt
4. Condiciones:
 - Debe ejecutarse continuamente (simulación de servicio activo).
 - No debe reprocesar el mismo mensaje.
 - Debe manejar archivos vacíos.

◆ Parte 2: Desarrollo del CLIENTE (1 hora)

Se desarrolla sin necesidad de ver el código del servidor

Actividad

1. Crear archivo:
cliente.py
2. Implementar un programa que:
 - Solicite un mensaje al usuario.
 - Escriba el mensaje en:
entrada_servidor.txt
 - Espere un tiempo (simulando latencia).
 - Lea la respuesta desde:
salida_servidor.txt
 - Muestre el resultado en pantalla.
3. Condiciones:
 - No debe asumir cómo funciona el servidor internamente.
 - Solo debe conocer los archivos de entrada y salida.

◆ Parte 3: Integración del sistema (25 minutos)

1. Ejecutar primero el servidor.

-
2. Ejecutar el cliente.
 3. Enviar varios mensajes.
 4. Verificar:
 - Flujo correcto
 - Respuesta adecuada

NOTA: los archivos de comunicación deben estar en una sola carpeta fuera de ambos proyectos

6. Resultados esperados:

Al finalizar la práctica, el estudiante entregará:

- Código fuente de servidor.py y cliente.py (repositorio Git).
- Capturas de pantalla que evidencien: (a) el archivo entrada_servidor.txt siendo llenado por el cliente, (b) salida_servidor.txt con el resultado procesado, (c) al menos 3 mensajes procesados correctamente.
- Respuestas a las preguntas de control en el reporte de práctica. Con ello demostrará: el desacoplamiento entre cliente y servidor, la comunicación indirecta por medio compartido, y las limitaciones del modelo (tiempo real, conflictos de escritura, dependencia del medio).

Desarrollo:

Primero se crean los tres cuerpos una para el servidor, otra para el cliente y una tercera carpeta para los archivos de comunicación, lo que esto permite mantener separados los componentes del sistema y demuestra el desacoplamiento entre cliente y servidor.

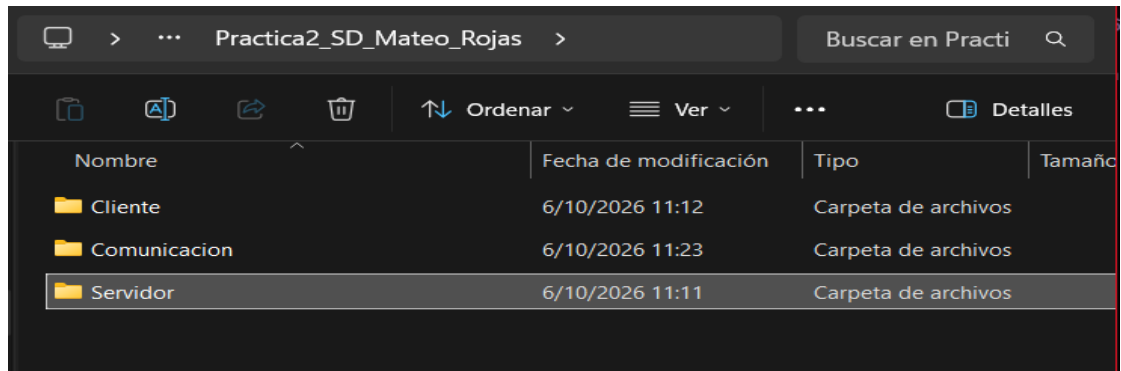


Fig.1 Carpetas de la estructura del proyecto.

Se crea el archivo que funciona como el medio mediante el cual el cliente envía la información al servidor, al principio se deja vacío ya cuando el usuario escribe un mensaje en el programa cliente, el mensaje se almacena en este archivo.

Dentro de la carpeta "comunicación" se crean los archivos:

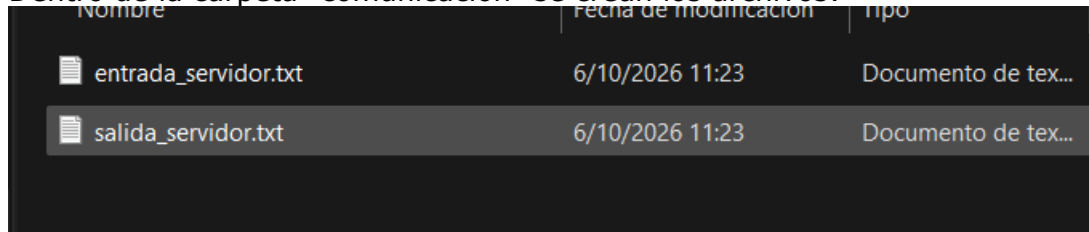
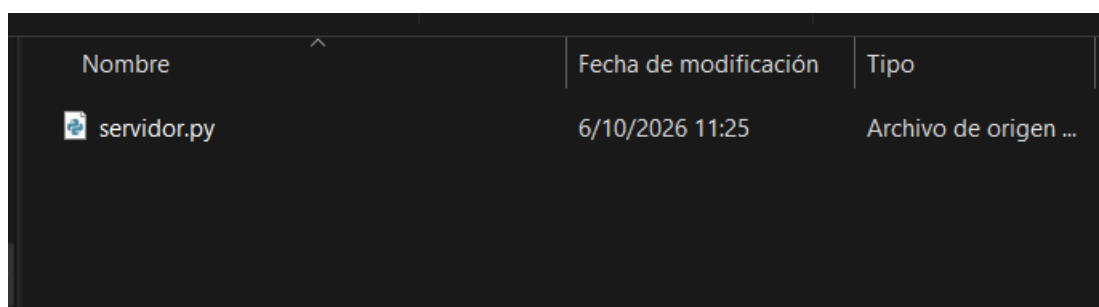


Fig.2 Nombres de los archivos.

El archivo entrada_servidor.txt es utilizado para almacenar el mensaje enviado por el cliente.

El archivo salida_servidor.txt contiene las respuestas generadas por el servidor.

Dentro de la carpeta servidor se crea el archivo servidor.py que es desarrollado para permanecer activo y revisar el archivo entrada_servidor.txt, cuando se detecta un nuevo mensaje se realiza el procesamiento y el resultado es almacenado en salida_servidor.txt.



Fi.3 Creacion de servidor.py

```
1  import time
2  from pathlib import Path
3
4
5  BASE = Path(__file__).resolve().parent.parent
6  ARCHIVO_ENTRADA = BASE / "comunicacion" / "entrada_servidor.txt"
7  ARCHIVO_SALIDA = BASE / "comunicacion" / "salida_servidor.txt"
8
9
10 mensajes_procesados = 0
11
12 print("=====")
13 print("      SERVIDOR INICIADO")
14 print("=====")
15 print("Esperando mensajes del cliente...\n")
16
17 while True:
18     try:
19         with open(ARCHIVO_ENTRADA, "r", encoding="utf-8") as archivo:
20             mensajes = archivo.readlines()
21
22             if len(mensajes) > mensajes_procesados:
23
24                 nuevos_mensajes = mensajes[mensajes_procesados:]
25
26                 for mensaje in nuevos_mensajes:
27
28                     mensaje = mensaje.strip()
29
30                     if mensaje != "":
31
32                         print("Mensaje recibido:", mensaje)
33
34                         respuesta = mensaje.upper()
35
36                         print("Mensaje procesado:", respuesta)
37
38                         with open(ARCHIVO_SALIDA, "a", encoding="utf-8") as archivo:
39                             archivo.write(respuesta + "\n")
40
41                         print("Respuesta enviada al cliente.\n")
42
43                         mensajes_procesados = len(mensajes)
44
45     except FileNotFoundError:
46         print("Esperando archivos de comunicación...")
47
48     time.sleep(1)
```

Fig.4 Código de servidor.py

Dentro de la carpeta cliente se crea el cliente.py que es desarrollado para permitir que el usuario introduzca un mensaje mediante el teclado.

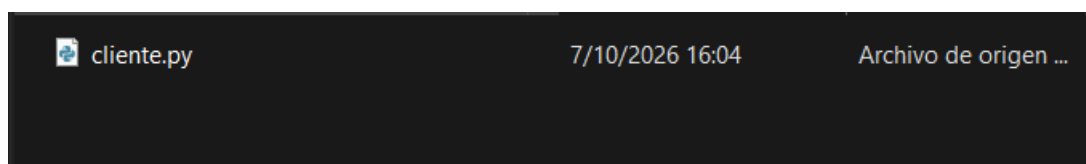


Fig. 5 Creación del cliente.py

El cual una vez ingresado el mensaje se escribe en entrada_servidor.txt, después el cliente espera un pequeño intervalo de tiempo para simular latencia y consulta con el archivo de salida_servidor.txt y muestra al usuario la respuesta generada por el servidor.

Se crea el cliente.py que solicita mensaje, escribirlo en entrada_servidor.txt que superan para simular latencia, que lee la salida_servidor.txt y muestran los resultados.

```
Practica2_SD_Mateo_Rojas > Cliente > cliente.py > ...
1  import os
2  import time
3
4  BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
5  ARCHIVO_ENTRADA = os.path.join(BASE_DIR, "comunicacion", "entrada_servidor.txt")
6  ARCHIVO_SALIDA = os.path.join(BASE_DIR, "comunicacion", "salida_servidor.txt")
7
8  print("=====")
9  print("          CLIENTE INICIADO")
10 print("=====")
11
12 while True:
13
14     mensaje = input("\nIngrese un mensaje (o 'salir' para terminar): ")
15
16     if mensaje.lower() == "salir":
17         print("Cliente finalizado.")
18         break
19
20     if mensaje.strip() == "":
21         print("No puede enviar un mensaje vacío.")
22         continue
23
24
25     try:
26         with open(ARCHIVO_SALIDA, "r", encoding="utf-8") as archivo:
27             respuestas_anteriores = archivo.readlines()
28     except FileNotFoundError:
29         respuestas_anteriores = []
30
31     cantidad_anterior = len(respuestas_anteriores)
32
33
```

```
with open(ARCHIVO_ENTRADA, "a", encoding="utf-8") as archivo:
    archivo.write(mensaje + "\n")

print("Mensaje enviado al servidor.")
print("Esperando respuesta...")

time.sleep(2)

respuesta_recibida = False

for intento in range(10):
    try:
        with open(ARCHIVO_SALIDA, "r", encoding="utf-8") as archivo:
            respuestas = archivo.readlines()

            if len(respuestas) > cantidad_anterior:
                respuesta = respuestas[-1].strip()

                print("\nRespuesta del servidor:")
                print(respuesta)

                respuesta_recibida = True
                break

    except FileNotFoundError:
        pass

    time.sleep(1)

if not respuesta_recibida:
    print("No se recibió respuesta del servidor.")
```

Fig.6 Código de cliente.py

Para realizar la integración del sistema se ejecuto primero el servidor.py que al ejecutarlo, el servidor queda activo esperando los mensajes provenientes del cliente.

```
PS C:\Users\Usuario iTC\Desktop\Sistemas Distribuidos\Practica2_SD_Mateo_Rojas\Servidor\Servidor> python servidor.py
Esperando archivos de comunicación...
Esperando archivos de comunicación...
Esperando archivos de comunicación...
Esperando archivos de comunicación...
Esperando archivos de comunicación...
Esperando archivos de comunicación...
Esperando archivos de comunicación...
Esperando archivos de comunicación...
Esperando archivos de comunicación...
Esperando archivos de comunicación...
```

Fig. 7 Servidor ejecutándose y esperando las solicitudes

Sin cerrar el servidor se abre otra terminal y se ejecuta cliente.py que muestra la opción para ingresar mensajes por el teclado.

```
PS C:\Users\Usuario iTC\Desktop\Sistemas Distribuidos\Practica2_SD_Mateo_Rojas\Cliente> python cliente.py
=====
CLIENTE INICIADO
=====

Ingrese un mensaje (o 'salir' para terminar):
```

```
PS C:\Users\Usuario iTC\Desktop\Sistemas Distribuidos\Practica2_SD_Mateo_Rojas\Cliente> python cliente.py
=====
CLIENTE INICIADO
=====

Ingrese un mensaje (o 'salir' para terminar): Hola servidor
Mensaje enviado al servidor.
Esperando respuesta...
```

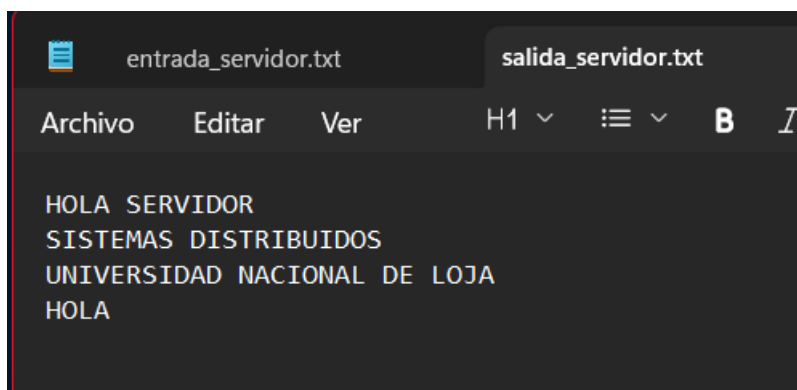
Fig.8 Ejecución de cliente.py

Se envia el mensaje de “Hola servidor” que este se almacena en entrada_servidor.txt

```
Hola servidor
Sistemas Distribuidos
Universidad Nacional de Loja
```

Fig.9 Mensajes recibidos en entrada_servidor.txt

Se observan los contenidos principalmente de entrada_servidor.txt en el cual se almacenan los mensajes que son enviados desde el cliente, estos archivos funcionan como medio de comunicación entre el cliente y el servidor.



```
entrada_servidor.txt salida_servidor.txt
Archivo Editar Ver H1 H1 B I
HOLA SERVIDOR
SISTEMAS DISTRIBUIDOS
UNIVERSIDAD NACIONAL DE LOJA
HOLA
```

Fig.10 Mensajes recibidos en salida_servidor.txt

Se observan los mensajes procesados por el servidor, estos textos son enviados por el cliente fueron convertidos a letras mayúsculas.

```
PS C:\Users\Usuario iTC\Desktop\Sistemas Distribuidos\Practica2_SD_Mateo_Rojas\Servidor> python servidor.py
Mensaje recibido: Hola servidor
Mensaje procesado: HOLA SERVIDOR
Respuesta enviada al cliente.
-----
Mensaje recibido: Sistemas Distribuidos
Mensaje procesado: SISTEMAS DISTRIBUIDOS
Respuesta enviada al cliente.
-----
Mensaje recibido: Universidad Nacional de Loja
Mensaje procesado: UNIVERSIDAD NACIONAL DE LOJA
Respuesta enviada al cliente.
```

Fig.11 Ejecución del servidor.py

Se ejecuto al servidor, el cual permaneció activo revisando los archivos de entrada, que detectan nuevos mensajes.

Los resultados nos demostraron que existió un flujo correcto entre cliente y servidor y que los archivos compartidos permitieron simular la comunicación entre ambos componentes.

7. Preguntas de Control:

¿Qué significa que cliente y servidor estén desacoplados?

Pueden funcionar y desarrollarse de manera independiente, el cliente no necesita conocer como procesa internamente la información al servidor, únicamente debe reconocer el mecanismo mediante el cual envía y recibe la información.

¿Qué ventajas tiene desarrollar componentes por separado?

Nos permite realizar modificaciones, mantenimiento y pruebas independientemente, facilitando el desarrollo del sistema por diferentes programadores.

¿Qué problemas pueden surgir al integrar sistemas independientes?

Pueden aparecer problemas de sincronización, retrasos, incompatibilidad en los datos con errores de lectura cuando varios procesos utilizan los mismos recursos.

¿Qué sucede si el servidor no está ejecutándose?

El cliente puede enviar el mensaje al archivo de entrada, pero no existe un proceso que lo analice y se genera una respuesta correspondiente.

¿Cómo se simula la comunicación en esta práctica?

Mediante los archivos de textos compartidos los cuales reciben las solicitudes de entrada_servidor.txt y salida_servidor.txt.

¿Qué limitaciones tiene este modelo frente a uno real?

La comunicación directa de la red puede existir problemas de sincronización con retrasos.

¿Cómo se podría mejorar la sincronización?

Para mejorar la sincronización se podrían utilizar identificadores para los mensajes que controlan el acceso a las colas de los mensajes.



¿Qué pasaría si múltiples clientes usan el sistema?

Varios clientes al intentar escribir al mismo tiempo en el mismo archivo puede generar conflictos que sería necesario determinar que respuesta perteneces a cada cliente.

8. Evaluación

NO APLICA

9. Bibliografía

Semana 2. Guaman, Jose. 2026



10. Elaboración y Aprobación

Elaborado por	José Guamán Docente	 Validar únicamente en FirmaEC. Firmado electrónicamente por: JOSE OSWALDO GUAMAN QUINCE
Ejecutado Por	Rene Rivera Técnico Docente	 Validar únicamente en FirmaEC. Firmado electrónicamente por: RAMIRO RENE RIVERA GUAMAN
Aprobado por	Edison L Coronel Romero Director de Carrera	 Firmado electrónicamente por: EDISON LEONARDO CORONEL ROMER Validar únicamente con FirmaEC